Analisis Algoritme 3

Slide disarikan dari berbagai sumber

Tujuan

- Mahasiswa dapat:
  - memahami laju pertumbuhan fungsi untuk kasus *worst case*, *best case*
  - Memahami kombinasi batas atas dan bawah untuk *average case*
  - Menganalisis perbandingan algoritme pada kasus terburuk, dan rata-rata
  - merancang algoritme sederhana yang memiliki kompleksitas baik

The main idea of asymptotic analysis

- to have a measure of efficiency of algorithms independently from machine specification
- Asymptotic notations represent time complexity of algorithms

LAJU PERTUMBUHAN FUNGSI
GROWTH RATE OF FUNCTION

- "Waktu" yang diperlukan oleh sebuah algoritme untuk memproses suatu masukan berukuran tertentu dapat direpresentasikan dalam sebuah fungsi y = f(x).
  - y = waktu atau jumlah langkah yang dibutuhkan
  - x = ukuran masukan yang diberikan.
- Menggambarkan perilaku fungsi pada ukuran input yang sangat besar.
- Fungsi ini dapat digunakan untuk membandingkan performa dua buah algoritme.

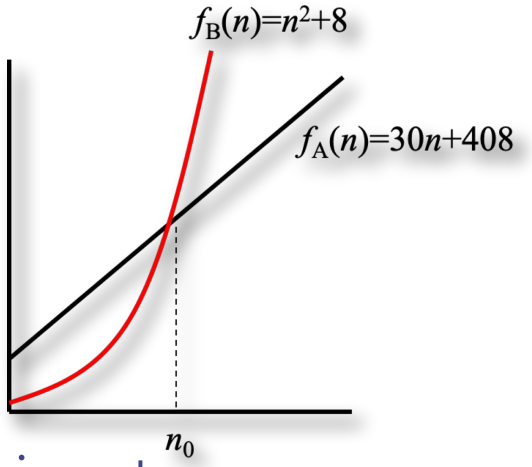
CONTOH
GROWTH RATE OF FUNCTION

- Misalkan Anda mengembangkan dua buah algoritme untuk memproses data berukuran n.
  - Algoritma A: $f_a(n) = 30n + 408$.
  - Algoritme B: $f_b(n) = n^2 + 8$.
- Mana yang lebih baik? Algoritme A atau B?

CONTOH
GROWTH RATE OF FUNCTION

- Dari ilustrasi berikut, terlihat bahwa $f_b(n) < f_a(n)$ hingga suatu titik n_0. Setelah itu, $f_b(n) > f_a(n)$.

Ukuran Input (n)	Algoritme A	Algoritme B
1	438	9
10	708	108
20	1008	408
100	3408	10008
200	6408	40008

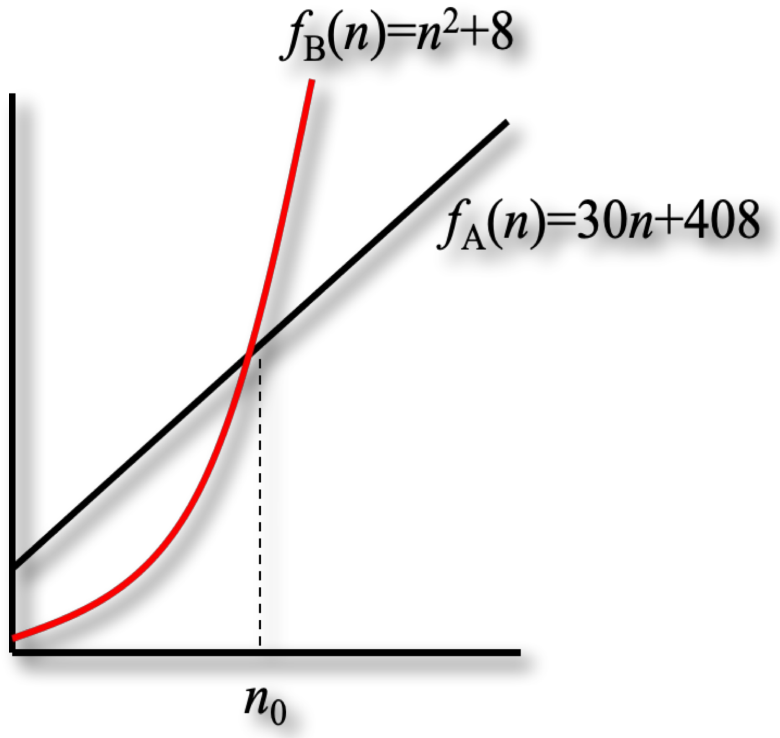


- Karena (lazimnya) perhatian kita adalah pada kasus dengan ukuran input besar, yang perlu kita perhatikan adalah bagian setelah $n > n_0$.

- Kecuali jika pada kasus yang diberikan, ukuran input-nya < n_0.

CONTOH
GROWTH RATE OF FUNCTION

- Berapakah nilai n_0 pada kasus tadi?

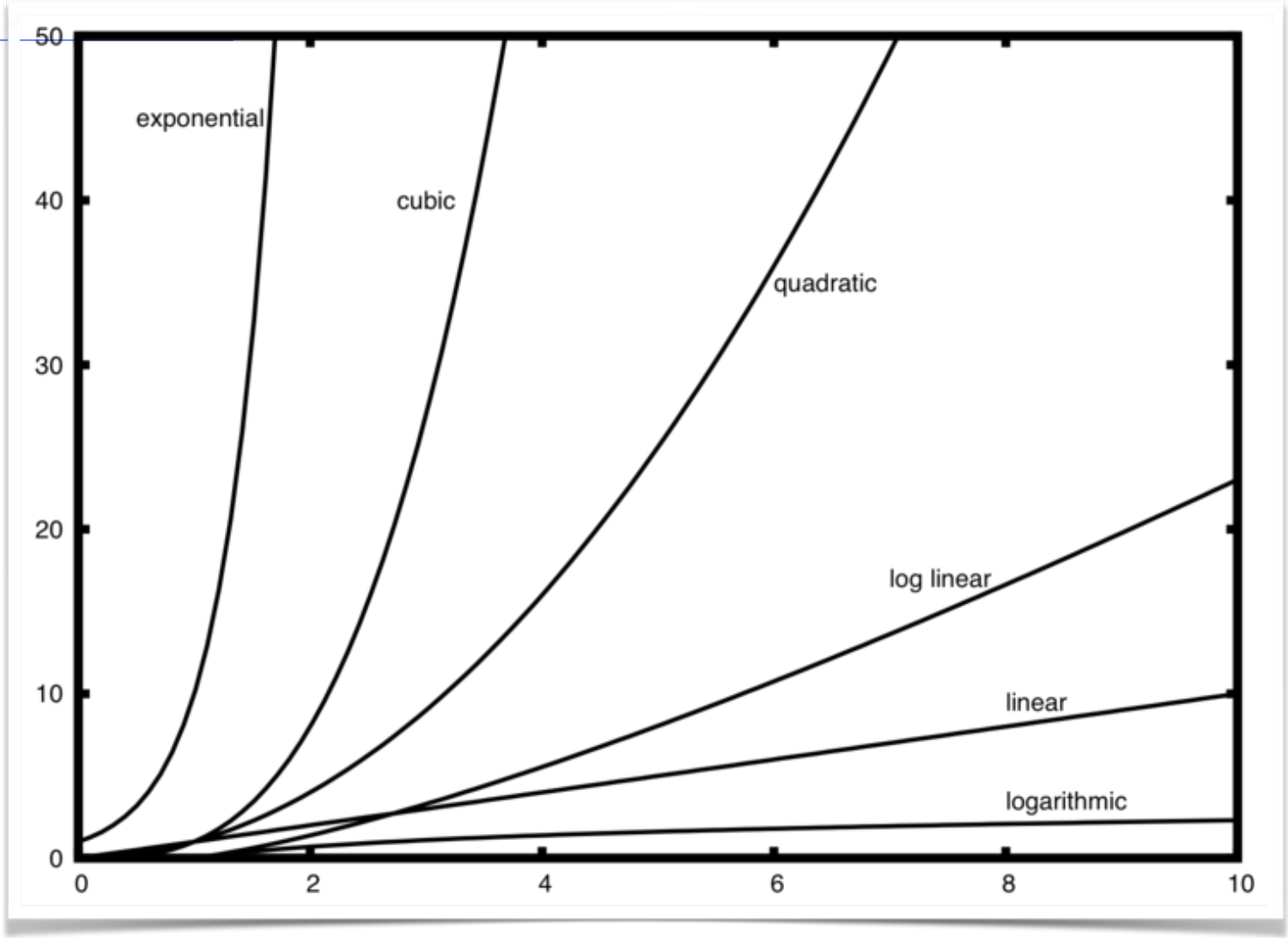


$$n^2 - 30n - 400 = (n + 10)(n - 40)$$

$$n_0 = 40$$

LAJU PERTUMBUHAN FUNGSI

GROWTH RATE OF FUNCTION



DEFINISI
NOTASI ASIMTOTIK

- Kita dapat menganalisis algoritme berdasarkan jumlah operasi yang dilakukannya yang merupakan fungsi dari jumlah input yang diberikan.
- Kita dapat memiliki tiga kasus untuk menganalisis algoritme: 1) Kasus Terburuk 2) Kasus Rata-rata 3) Kasus Terbaik
- Notasi asimtotik : Big Oh, Big Theta, Big Omega

DEFINISI

BEST CASE, AVERAGE CASE, DAN WORST CASE

- Kasus Terbaik/Best Case: kondisi yang mengakibatkan jumlah langkah yang dilakukan minimum. Biasanya tidak terlalu bermanfaat untuk dihitung.
- Kasus Rata-rata/Average Case: rata-rata jumlah langkah yang dilakukan untuk seluruh kemungkinan input. Biasanya cukup sulit untuk dihitung.
- Kasus terburuk/Worst Case: kondisi yang mengakibatkan jumlah langkah yang dilakukan maksimum. Sangat sering digunakan untuk menganalisis algoritme.

DEFINISI

BEST CASE, AVERAGE CASE, DAN WORST CASE

- Identifikasi best, average, dan worst case dari sequential search. Mencari target pada data berukuran size dari elemen pertama sampai data ditemukan atau seluruh data dicek.

```
1   int search(int data[], int size, int target)
2   {
3       int i;
4       for (i=0; i<size; i++)
5       {
6           if (data[i] == target)
7               return i;
8       }
9       return -1;
10  }
```

Complexity is Simple

- Simple way to analyze the algorithms: calculate complexity of algorithms (asymptotic analysis)
- We can have three cases to analyze an algorithm:
  1) Worst Case
  2) Average Case
  3) Best Case

BEST CASE

- Target terdapat di elemen pertama. Berapapun ukuran input, hanya diperlukan 1 kali pengecekan.

WORST CASE

- Target tidak ada. Seluruh elemen data dicek dan memerlukan sebanyak n kali pengecekan.

AVERAGE CASE

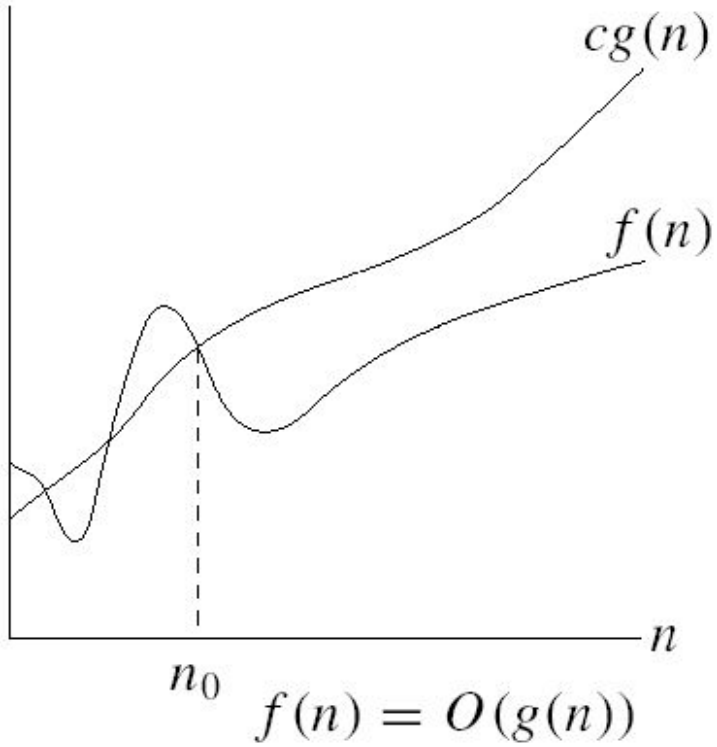
- Jumlah pengecekan bergantung pada elemen mana target berada.
- Misalkan distribusi lokasi target mengikuti sebaran uniform.
- Maka kasus rata-rata dapat dihitung dengan mencari nilai harapan dari lokasi kemunculan target (berikut dengan kondisi target tidak ada dalam data).
- Jumlah operasinya sebanyak $(1+2+3\ldots+n+1) / (n+1) \approx n$
- Menghitung kasus rata-rata bisa jadi cukup sulit bergantung pada masalah yang diberikan.

Worst Case

- upper bound on the resources required by the algorithm.
- indicates the longest running time performed by an algorithm
- For Linear Search, when the worst case happens ?
  - when the element to be searched (x in the above code) is not present in the array
  - it takes O(*n*) steps to search

Worst case ☐ Big-O notation

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\} .$$



- f(n) = O(g(n)) menyatakan bahwa f(n) tumbuh lebih lambat dibandingkan suatu hasil kali konstan dari g(n) pada saat n membesar tanpa batas.
- f(n) = O(g(n)) dapat dibuktikan secara matematis dengan mencari c dan n_0.
- f(n) = O(g(n)) juga dapat ditulis sebagai $f(n) \in O(g(n))$

Worst case ☐ Big-O notation (cont.)

- Tunjukkan bahwa running time 30n+8 adalah O(n).

→ $30n + 8 \leq cn$

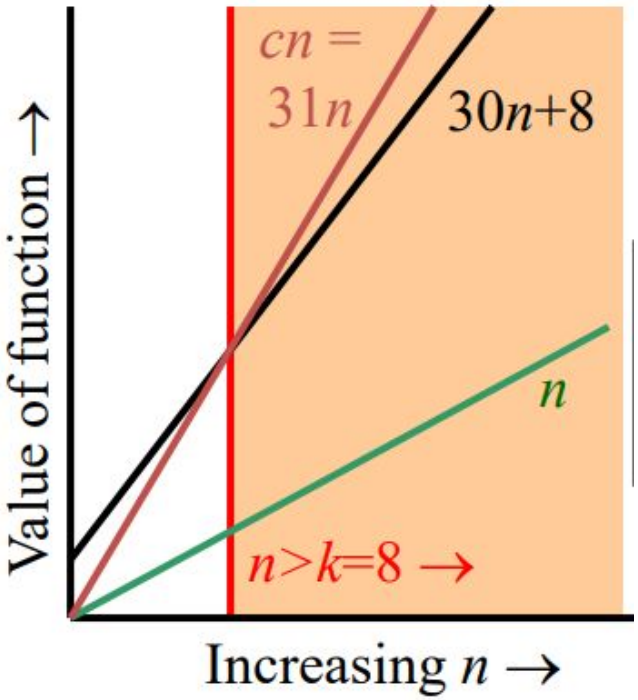
→ $\frac{30n+8}{n} \leq c$

→ $30 + \frac{8}{n} \leq c$

→ $n = 8, c = 31$

→ running time 30n+8 adalah O(n)

→ terbukti



Big-O Proof

1. Tunjukkan bahwa kompleksitas n^2+1 adalah $O(n^2)$.
   - Tunjukkan $\exists c, n_0 : \forall n > n_0 : n^2+1 \leq cn^2$.

$\rightarrow \frac{n^2+1}{n^2} \leq c \rightarrow 1 + \frac{1}{n^2} \leq c \rightarrow$ c=2, n=1

$\rightarrow n^2+1 \leq 2n^2$ untuk $n \geq 1 \rightarrow$ terbukti

2. Tunjukkan bahwa $T(n) = 2n^2 + 6n + 1 = O(n^2)$.
   - Tunjukkan $\exists c, n_0 : \forall n > n_0 : 2n^2 + 6n + 1 \leq cn^2$.

$\rightarrow 2n^2 + 6n + 1 \leq cn^2 \rightarrow 2 + \frac{6}{n} + \frac{1}{n^2} \leq c$

$\rightarrow n = 1, c = 9$

$\rightarrow 2n^2 + 6n + 1 \leq 9n^2$ untuk $n \geq 1 \rightarrow$ terbukti

Sifat-sifat Big-O

- Hukum Penjumlahan
  - O(f + g) = max {O(f), O(g)}
- Hukum Perkalian
  - O(f · g) = O(f) · O(g)

```
void method3(int [] arr) {
   O(n) for(int i = 0; i <arr.length; i++)
        {
              method1(arr); O(n)
              method2(arr); O(n^2)
        }
}
```

$O(n^3)$

Sifat-sifat Big-O

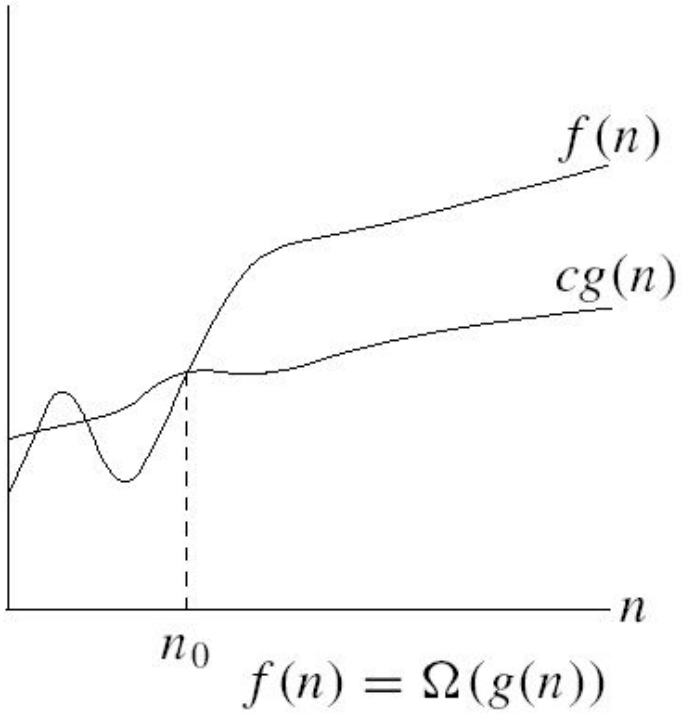
- Transitivity Example
  - if g = O(f) and f = O(h) then g = O(h)
  - Contoh: Tunjukkan $\frac{5}{2}n \leq 5n^2$
    - $\frac{5}{2} \leq 5,\ maka\ \frac{5}{2}n \leq 5n$
    - Theorem: if a = b and 0 ≤ c ≤ d then a*c ≤ b*d.
    - $5n \leq 5n^2$ → $\frac{5}{2}n \leq 5n^2$ transitivity hold

Best Case

- Lower bound on the resources required by the algorithm.
- indicates the fastest running time performed by an algorithm
- For Linear Search, when the best case happens ?
  - when x is present at the first location
  - it takes O(1) to search

Best case ☐ Big-Omega notation

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\}.$$



f(x) adalah Ω(g(x)) menyatakan bahwa f(x) tumbuh lebih lambat dibandingkan suatu hasil kali konstan dari g(x) pada saat x membesar tanpa batas.

Best case ☐ Big-Omega notation

- Tunjukkan bahwa running time dari $T(n) = n^3 + 20n$ adalah $\Omega(n^2)$

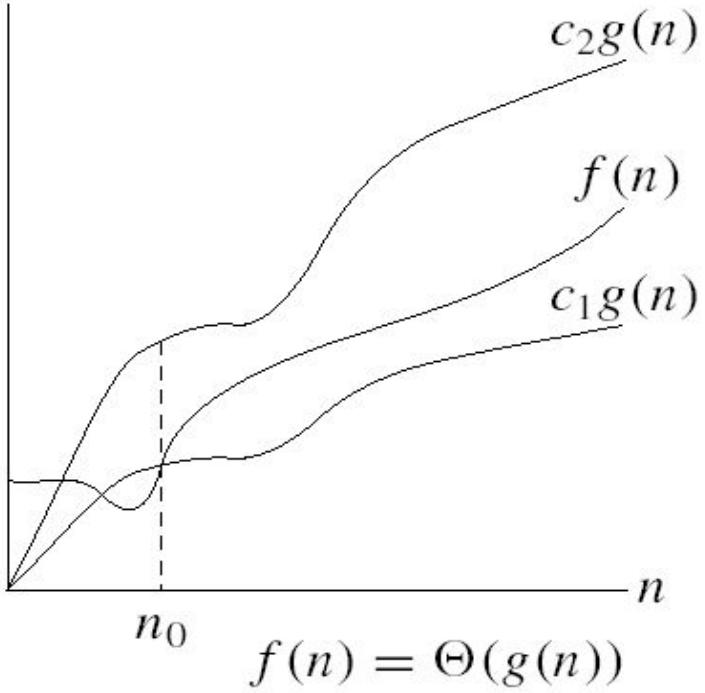
→ $n^3 + 20n \geq cn^2$

→ For c=1, $n^3 + 20n \geq n^3$ → $n^3 + 20n \geq n^3 \geq n^2$ (transitivity)

→ c=1, n≥1

Average Case

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\}.^1$$



the average-case complexity of insertion sort is taking into account the total number of data moves is at least zero and at most the number of comparisons

Average Case

- indicates the average running time performed by an algorithm
- For Linear Search, how to calculate the average case?

LATIHAN 1

GROWTH RATE OF FUNCTION

- Diberikan dua buah algoritme A dan B. Berapakah nilai n_0 ?.

$f_a(n)$	$f_b(n)$	n_0
$n^2 + 90$	$16n + 42$	
$2n^2 + 20$	$11n + 6$	

1. Tunjukkan running time dari $T(n) = n^3 + 20n + 1$ adalah $O(n^3)$.
2. Tunjukkan running time dari $T(n) = n^3 + 20n + 1$ adalah $O(n^4)$.
3. Tunjukkan running time dari $T(n) = n^3 + 20n + 1$ adalah bukan $O(n^2)$.

LATIHAN 2

BEST CASE, AVERAGE CASE, DAN WORST CASE

- Identifikasi best, average, dan worst case dari algoritme bubble sort berikut.

```
1   procedure bubbleSort( list : array of items )
2      loop = list.count;
3
4      for i = 0 to loop-1 do:
5         for j = 0 to loop-1 do:
6            if list[j] > list[j+1] then
7               swap( list[j], list[j+1] )
8               swapped = true
9            end if
10        end for
11     end for
12  end procedure return list
```

1. Tunjukkan running time dari $T(n) = n^3 + 20n + 1$ adalah $O(n^3)$

Untuk menunjukkan bahwa $T(n) = n^3 + 20n + 1$ adalah $O(n^3)$, kita perlu menunjukkan bahwa terdapat konstanta $c > 0$ dan $n_0 \geq 0$ sehingga:

$$T(n) \leq c \cdot n^3 \quad \text{untuk semua } n \geq n_0$$

Diketahui:

$$T(n) = n^3 + 20n + 1$$

Untuk nilai n yang besar, suku n^3 mendominasi suku-suku lainnya. Oleh karena itu:

$$T(n) \leq n^3 + 20n^3 + 1n^3 = 21n^3$$

Maka, kita dapat memilih $c = 21$ dan $n_0 = 1$, sehingga:

$$T(n) \leq 21n^3 \quad \text{untuk semua } n \geq 1$$

Dengan demikian, $T(n)$ adalah $O(n^3)$.

2. Tunjukkan running time dari $T(n) = n^3 + 20n + 1$ adalah $O(n^4)$

Untuk menunjukkan bahwa $T(n) = n^3 + 20n + 1$ adalah $O(n^4)$, kita perlu menunjukkan bahwa terdapat konstanta $c > 0$ dan $n_0 \geq 0$ sehingga:

$$T(n) \leq c \cdot n^4 \quad \text{untuk semua } n \geq n_0$$

Diketahui:

$$T(n) = n^3 + 20n + 1$$

Untuk nilai n yang besar:

$$T(n) = n^3 + 20n + 1 \leq n^4$$

Sebagai contoh, dengan memilih $c = 1$ dan $n_0 = 1$, kita memiliki:

$$T(n) \leq 1 \cdot n^4 \quad \text{untuk semua } n \geq 1$$

Dengan demikian, $T(n)$ adalah $O(n^4)$.

3. Tunjukkan running time dari $T(n) = n^3 + 20n + 1$ adalah bukan $O(n^2)$

Untuk menunjukkan bahwa $T(n) = n^3 + 20n + 1$ **bukan** $O(n^2)$, kita perlu menunjukkan bahwa tidak ada konstanta $c > 0$ dan $n_0 \geq 0$ yang memenuhi:

$$T(n) \leq c \cdot n^2 \quad \text{untuk semua } n \geq n_0$$

Diketahui:

$$T(n) = n^3 + 20n + 1$$

Untuk nilai n yang besar, n^3 mendominasi n^2. Ini berarti:

$$T(n) = n^3 + 20n + 1 > c \cdot n^2 \quad \text{untuk } n \text{ besar}$$

Tidak peduli berapa pun nilai konstanta c yang dipilih, n^3 pada akhirnya akan lebih besar dari $c \cdot n^2$ seiring bertambahnya n. Oleh karena itu, $T(n)$ **bukan** $O(n^2)$.

Algoritme **Bubble Sort** yang ditampilkan memiliki tiga skenario kasus yang berbeda dalam hal kompleksitas waktu: **best case**, **average case**, dan **worst case**. Mari kita bahas masing-masing:

1. Best Case

- **Kondisi:** Daftar sudah terurut secara ascending.
- **Kompleksitas:** $O(n)$
- **Penjelasan:** Pada kasus terbaik, ketika daftar sudah terurut, setiap elemen dibandingkan tetapi tidak ada swap yang terjadi. Algoritme bisa berhenti lebih awal jika implementasinya dioptimalkan untuk mendeteksi bahwa tidak ada swap yang dilakukan dalam satu iterasi. Namun, pada kode yang ditunjukkan, tidak ada pengecekan apakah swap terjadi atau tidak, sehingga tetap melakukan iterasi lengkap meskipun daftar sudah terurut.

2. **Average Case**

- **Kondisi:** Daftar diisi dengan elemen yang teracak.
- **Kompleksitas:** $O(n^2)$
- **Penjelasan:** Pada kasus rata-rata, elemen-elemen dalam daftar berada dalam urutan acak. Algoritme akan memerlukan beberapa iterasi untuk menukar elemen-elemen sampai daftar menjadi terurut. Setiap iterasi dalam loop luar melakukan perbandingan dan kemungkinan swap pada setiap elemen, yang mengarah pada kompleksitas kuadratik.

3. **Worst Case**

- **Kondisi:** Daftar terurut secara descending (terbalik dari urutan yang diinginkan).
- **Kompleksitas:** $O(n^2)$
- **Penjelasan:** Pada kasus terburuk, setiap elemen berada pada posisi yang salah, sehingga algoritme perlu melakukan swap untuk setiap elemen pada setiap iterasi. Ini menyebabkan algoritme menjalankan iterasi maksimum dari loop, membuatnya sangat tidak efisien dengan kompleksitas waktu $O(n^2)$.

Kesimpulan:

- Best Case: $O(n)$ jika algoritme dioptimalkan untuk berhenti lebih awal.
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$